

Homigo Care iOS Application Blueprint

- [HOMIGO CARE — iOS APPLICATION](#)
 - [Complete Native Build Blueprint for an AI Coding Agent \(Manus AI\)](#)
 - [Repository, Infrastructure, Design System, Screen-by-Screen Specification](#)
 - [0. Purpose, Working Method & When to Ask Questions](#)
 - [1. Repository & Infrastructure Setup](#)
 - [1.1 GitHub Repository — Create or Reuse](#)
 - [1.2 Firebase — Add the iOS App to the Same Project](#)
 - [1.3 Supabase — Same Project, Swift Client](#)
 - [1.4 Cloudinary — Same Account, Same Presets](#)
 - [1.5 Admin Panel Connection \(Vercel MCP — read/verify only\)](#)
 - [1.6 Secrets & Environment Configuration](#)
 - [1.7 Founder-Provided Assets — Exact Drop-In Paths](#)
 - [1.8 AI Image-Generation Request Pipeline — 01_ai_images.json](#)
 - [2. Global Design System — Single Source of Truth \(SwiftUI\)](#)
 - [3. Technology Stack \(iOS — final, authoritative\)](#)
 - [4. Icons — What's Coded vs. What's AI-Generated \(decision rule\)](#)
 - [5. Splash Screen & Video Integration](#)
 - [6. Authentication & OTP Flow \(via the Admin Panel's EmailJS SMTP\)](#)
 - [7. Complete Screen-by-Screen Blueprint](#)
 - [7.1 Shared — Splash, Onboarding, Auth, Role Selection \(9 screens\)](#)
 - [7.2 Patient Flow — 32 Screens \(P-01-P-32\)](#)
 - [7.3 Professional Flow — 15 Screens \(PR-01-PR-15\)](#)
 - [8. App-Wide Navigation Map \(AppRouter.swift, NavigationStack-based\)](#)
 - [9. Data Sync Architecture \(iOS-side summary — identical backend to Android\)](#)
 - [10. Build & Release Pipeline — Real Signed Build Requests](#)
 - [11. Final QA Checklist Before Calling the Build "Done"](#)

HOMIGO CARE — iOS APPLICATION

Complete Native Build Blueprint for an AI Coding Agent (Manus AI)

Repository, Infrastructure, Design System, Screen-by-Screen Specification

0. Purpose, Working Method & When to Ask Questions

This is a **single, self-contained build order** for the **official native iOS application** of Homigo Care — the **premium-tier** sibling of the Android app, sharing the same backend but with its own independent Swift/SwiftUI codebase, its own repository, and a visual polish level that should feel noticeably more “Apple-native” than the Android build (large-title navigation, native blur materials, spring animations, haptics).

MCP servers available to the agent: GitHub, Supabase, Firebase CLI, Vercel (read-only, admin panel awareness), Cloudinary — identical set to the Android project.

Working method — build in phases, not one giant commit: 1. **Phase 0** — repository + infrastructure wiring (\$1), empty app that builds and launches. 2. **Phase 1** — Shared auth/onboarding screens (\$7.1) + design system (\$2) fully working end-to-end against real Firebase/Supabase. 3. **Phase 2** — Full Patient flow (\$7.2), screen by screen, in the order listed, committing after each working screen. 4. **Phase 3** — Full Professional flow (\$7.3), same incremental approach. 5. **Phase 4** — Polish pass: haptics, animations, empty states, accessibility, then a real signed build request (\$10).

Commit and push after each phase (or each screen, if that's more reviewable) rather than attempting the entire app in one pass — this keeps the repository always in a real, connectable, buildable state that the founder can check at any point.

When information is missing, ask — don't guess. If a required asset (logo, splash video, a specific AI-generated PNG not yet supplied), a credential (Firebase/Supabase/Cloudinary key), or a product decision isn't available yet, the agent should pause and explicitly request it rather than inventing a placeholder that later needs to be found and removed. This applies especially to anything in \$1.7 (founder-provided assets) and \$6 (backend credentials).

1. Repository & Infrastructure Setup

1.1 GitHub Repository — Create or Reuse

1. Using the **GitHub MCP**, check whether a repository named exactly `HOMIGO-CARE-IOS` already exists under the connected account.

2. **If it does not exist:** create it fresh (private, Swift .gitignore, LICENSE per founder preference — default proprietary/all-rights-reserved).
3. **If it already exists:** do **not** build on top of whatever is currently inside it. Instead:
 - Delete all existing tracked source files (everything except .git, README.md if it should be rewritten anyway, and any CI secrets configuration which lives in GitHub settings, not in the tree).
 - Commit that removal as a clean “Reset repository for new HomigoCare iOS build” commit.
 - Then proceed to build the fresh structure below from scratch. This ensures no legacy/experimental code interferes with this specification.
4. Create this folder structure and commit it before writing feature code:

```
HOMIGO-CARE-IOS/
├── HomigoCare.xcodeproj/ (or HomigoCare.xcworkspace if SPM packages are split out)
├── HomigoCare/
│   ├── App/ # HomigoCareApp.swift (entry point), AppDelegate bridge for FCM
│   ├── Core/
│   │   ├── Theme/ # Color+Homigo.swift, Typography.swift, Spacing.swift, Shapes.swift
│   │   └── Constants.swift
│   ├── Data/
│   │   ├── Firebase/ # Auth, Firestore, FCM wrappers
│   │   ├── Supabase/ # Supabase Swift SDK repositories
│   │   ├── Cloudinary/ # Signed upload + fetch helpers
│   │   └── Repositories/ # Combines the above per feature
│   ├── Domain/ # Models, use cases
│   ├── Features/
│   │   ├── Shared/ # Splash, Onboarding, Auth, RoleSelect (SwiftUI Views + ViewModels)
│   │   ├── Patient/ # All P-xx screens
│   │   └── Professional/ # All PR-xx screens
│   ├── Components/ # Shared reusable SwiftUI views (buttons, cards, badges)
│   ├── Navigation/ # AppRouter.swift (NavigationStack-based)
│   ├── Resources/
│   │   ├── Assets.xcassets/ # App icon set, color set, AI-generated PNGs, launch image fallback
│   │   ├── SplashVideo.mp4 # founder-provided
│   │   └── Localizable.strings / ur.lproj, en.lproj
│   ├── Info.plist
│   └── GoogleService-Info.plist # Firebase config (added in $1.2)
├── assets_pending/
│   └── 01_ai_images.json # AI image-generation request file – see $1.8 (independent from the Android repo's own file)
├── .github/workflows/
│   └── ios-release.yml # CI: build signed .ipa on a macOS runner (see $10)
├── docs/
│   └── this-blueprint.pdf
└── README.md
```

1.2 Firebase — Add the iOS App to the Same Project

The iOS app joins the **same** Firebase project as the Android app (homigo-care-prod / homigo-care-dev) — one backend, two native clients, exactly as specified in the master architecture document.

1. In Firebase Console (via Firebase CLI MCP), register a new **iOS app** with bundle ID `pk.homigocare.ios`.
2. Download `GoogleService-Info.plist` and place it at `HomigoCare/GoogleService-Info.plist`.
3. Add these Firebase iOS SDKs via **Swift Package Manager**: `FirebaseAuth`, `FirebaseFirestore`, `FirebaseMessaging`, (optional) `FirebaseAnalytics`.
4. Enable **Push Notifications** and **Background Modes → Remote notifications** capabilities in the Xcode project so FCM can bridge to APNs correctly. This requires an Apple Push Notification key/certificate registered in the Apple Developer account (see the honest cost note in \$10).
5. Firestore usage rule is identical to Android: **only** real-time/live collections (`presence`, `bookings_live`, `chat_threads`, `notifications_feed`, `booking_status_events`) — everything structured goes to Supabase.

1.3 Supabase — Same Project, Swift Client

1. Point to the **same** Supabase project created for Android (homigo-care) — do not create a second database; both apps and the Admin Panel share one source of truth.
2. Add the **Supabase Swift SDK** via Swift Package Manager (`supabase-swift`: `Auth`, `PostgREST`, `Storage` modules).
3. Bridge the Firebase ID token to a Supabase session exactly as the Android app does (\$1.3 of the Android blueprint) — same JWT-claim-mapping function on the database side serves both clients identically, so a user who signs up on Android and later opens the iOS app (or vice versa) sees the same account and data.
4. Row Level Security policies are already defined at the database level (from the Android setup) and apply automatically — no iOS-specific RLS work needed, just correct auth wiring.

1.4 Cloudinary — Same Account, Same Presets

Reuse the exact folders and presets created for Android (\$1.4 of the Android blueprint): `homigo_public_assets` (unsigned, for app icon/illustration assets) and `homigo_user_documents` (signed, for CNIC/degree/prescription/receipt uploads). Use the **Cloudinary iOS SDK** (or plain `URLSession` multipart upload if the SDK adds unnecessary weight) with the same signing flow — request a signature from the Admin Panel’s Vercel API before any signed upload, never embed the API secret in the iOS app.

1.5 Admin Panel Connection (Vercel MCP — read/verify only)

Identical role to the Android setup: the iOS app is a **client** of the already-deployed Admin Panel API at `ADMIN_API_BASE_URL` (e.g. `https://admin.homigocare.pk/api`). Same two endpoint families matter here: - `POST /api/otp/send`, `POST /api/otp/verify` — see §6. - `GET /api/cms/:key` — support numbers, About Us, FAQ, legal pages, live pricing — **never hardcode these strings in Swift**, always fetch (with a cached fallback in `UserDefaults`/a small local store for offline viewing).

Confirm real connectivity to this same Admin Panel and the same Firebase/Supabase/Cloudinary projects that the Android app uses — this is one shared backend serving three real front-ends, not a separate iOS-only backend.

1.6 Secrets & Environment Configuration

Use an `.xcconfig`-based approach (`Secrets.xcconfig`, git-ignored, with a committed `Secrets.xcconfig.example` template) holding:

```
SUPABASE_URL = ...
SUPABASE_ANON_KEY = ...
CLOUDINARY_CLOUD_NAME = ...
CLOUDINARY_UNSIGNED_PRESET = homigo_public_assets
ADMIN_API_BASE_URL = https://admin.homigocare.pk/api
MAPS_PROVIDER = mapkit
```

Expose these to Swift via `Bundle.main.object(forKey:)` reading from `Info.plist` keys populated by the `.xcconfig` build settings. `GoogleService-Info.plist` is committed as normal per Firebase’s own convention (not a true secret for a client app).

1.7 Founder-Provided Assets — Exact Drop-In Paths

Asset	Path	Notes
App logo (master)	<code>Resources/Assets.xcassets/AppIcon.appiconset/</code>	Generate the full iOS App Icon set (all required sizes, no transparency, no rounded corners baked in — iOS masks it automatically) from the one master PNG using Xcode’s asset catalog tooling; never redraw the logo
Splash video	<code>Resources/SplashVideo.mp4</code>	See §5
This PDF	<code>docs/this-blueprint.pdf</code>	Reference copy for any other agent touching this repo

If any of these three have not been supplied yet when the agent reaches that phase, **stop and ask the founder for it** rather than shipping a placeholder logo or skipping the splash video.

1.8 AI Image-Generation Request Pipeline — `01_ai_images.json`

Same rules and JSON schema as the Android repository’s file (see the Android Blueprint §1.8) — **this is a separate file living in this iOS repo** (`assets_pending/01_ai_images.json`), because destination paths differ (they point into `Assets.xcassets` here, not `res/drawable`). Rules recap: - 20-50 image entries per file (create `02_ai_images.json` etc. beyond that). - Every entry specifies exact subject, background type (`transparent/solid_white/solid_brand_teal`), art style locked to the brand’s flat-vector teal/orange language, and an exact destination such as `Resources/Assets.xcassets/Illustrations/il_onboarding_homecare.imageset/`. - After the founder generates and places the PNGs (as a 1x/2x/3x set inside the relevant `.imageset` folder, or a single high-res image if using `Image("...")` from a flat asset), the agent wires each into its SwiftUI screen via `Image("il_onboarding_homecare")`. - Since iOS screens are visually the premium tier, prefer requesting slightly higher-fidelity/more polished versions of the same illustrations already defined for Android in the main brand blueprint — same content and palette, just note `"quality_tier": "premium"` in the JSON entry so the founder can ask the image model for extra polish where it matters (onboarding art, empty states, achievement badges).

2. Global Design System — Single Source of Truth (SwiftUI)

Create these first, before any screen. No SwiftUI View may hardcode a `Color(red:green:blue:)` literal, a raw font size, or a raw corner radius — everything routes through this layer.

Core/Theme/Color+Homigo.swift — define as a `Color Set` in `Assets.xcassets` (so both light-mode values and any future dark-mode override live in one managed place) **and** mirror as static `Color` constants for convenience:

```
extension Color {
    static let hgPrimaryTeal      = Color("PrimaryTeal")      // #1B6B63
    static let hgDarkTeal        = Color("DarkTeal")          // #0D4A44
    static let hgHeaderEmerald    = Color("HeaderEmerald")     // #0B5545 – premium header bars only
    static let hgSecondaryOrange = Color("SecondaryOrange")    // #F5823A
```

```

static let hgLightGray      = Color("LightGray")           // #F5F5F5 – input backgrounds
static let hgDarkGray      = Color("DarkGray")             // #333333 – body text
static let hgMediumGray    = Color("MediumGray")           // #888888 – secondary text
static let hgErrorRed       = Color("ErrorRed")             // #E53935 – SOS + errors only
static let hgSuccessGreen   = Color("SuccessGreen")         // #43A047
static let hgGoldElite     = Color("GoldElite")             // #D4AF37 – Elite badge only
static let hgSilverTopRated = Color("SilverTopRated")       // #C0C0C0 – Top Rated badge only
}

```

Core/Theme/Shapes.swift — Cards: RoundedRectangle(cornerRadius: 15, style: .continuous) (the .continuous style gives the native “squircle” iOS look — use it everywhere, this is a key part of the premium feel) with .shadow(radius: 3, y: 2) at low opacity. Buttons & inputs: RoundedRectangle(cornerRadius: 25, style: .continuous). SOS button: Circle(), always .hgErrorRed, pulsing via .scaleEffect inside a withAnimation(.easeInOut(duration: 1).repeatForever(autoreverses: true)).

Core/Theme/Typography.swift — Base font: **SF Pro** (the system default — do not override with a custom font; this is precisely what makes the app feel authentically native). Use Dynamic Type text styles wrapped in brand-consistent modifiers rather than fixed point sizes: .largeTitle for hero headers, .title2 (≈22pt) mapped to “H1,” .headline (≈18pt semibold) mapped to “H2,” .body (≈14–17pt) for body copy, .caption (≈12pt) for secondary text. Urdu text automatically switches to **Noto Nastaliq Urdu** (bundled .ttf, registered in Info.plist under UIAppFonts) when the app’s language is set to Urdu.

Core/Theme/Theme.swift — Light mode only for v1 (explicit founder decision — do **not** implement a .preferredColorScheme(.dark) variant yet, and do not let the app follow system dark mode automatically either; force .light).

Spacing scale: 4, 8, 12, 16, 24, 32 pt, screen horizontal padding always 16pt. Minimum tap target 44×44pt (Apple’s own HIG minimum — slightly smaller than Android’s 48dp because iOS’s own guideline is 44pt; still generous and satisfies the “clickable and comfortable” requirement).

Native materials — use these to earn the “premium” feel: .background(.ultraThinMaterial) for the tab bar and any floating panel, .navigationBarTitleDisplayMode(.large) on top-level screens (Home, Health Passport, Wallet), UIImpactFeedbackGenerator(style: .medium).impactOccurred() on every primary action (booking confirm, SOS hold-trigger, PIN success, Accept booking).

3. Technology Stack (iOS — final, authoritative)

Layer	Choice	Reason
Language	Swift (100%, latest stable version supported by the Xcode toolchain in use)	Apple’s official language
UI	SwiftUI	Apple’s official declarative UI, matches current iOS design language natively (continuous corners, native blur, spring animations) — this is what makes the app feel authentically premium-iOS, not a ported Android layout
Architecture	MVVM (ObservableObject/@Observable view models) + Clean layering (Data/Domain/Features)	Standard modern SwiftUI architecture, mirrors the Android app’s layering for easy cross-reference
Async	Swift Concurrency (async/await, Task, AsyncStream for Firestore listeners)	Native, no third-party reactive dependency needed
Navigation	NavigationStack (iOS 16+) with a typed AppRoute enum	Modern, deep-link-friendly, matches the Android nav-graph structure conceptually
DI	Lightweight manual DI via an AppContainer struct, or Factory/swift-dependencies if the agent prefers a library	Keeps the app dependency-light; avoids over-engineering for a v1
Local cache	SwiftData (iOS 17+) or Core Data fallback for older minimum-target iOS versions	Offline booking history, cached Health Passport, cached CMS content
Networking	URLSession (async/await) or Alamofire if preferred	Calls to the Admin Panel’s Vercel API (OTP, CMS)
Realtime backend	Firebase iOS SDK (Firestore + Cloud Messaging via SPM)	Live tracking, booking status, push notifications — same backend as Android
Structured backend	Supabase Swift SDK (Auth, PostgREST, Storage) Firebase Auth (Email/Password, “Sign Apple explicitly expects apps offering	Bookings, profiles, documents, payments, reviews

Auth	in with Google,” Anonymous/Guest) — also offer Sign in with Apple as an additional native-feeling option since this is the premium iOS build	third-party login (Google) to also offer Sign in with Apple for App Store approval — build this in from the start to avoid a review rejection later
Images	AsyncImage (built-in) or Kingfisher for more control/caching	Loads Cloudinary-hosted PNGs/illustrations
Maps	MapKit (native, free, zero API key)	The “premium native” choice — feels like a first-party iOS app; matches MAPS_PROVIDER = mapkit from \$1.6
Video (splash)	AVKit / AVPlayer	Native, smooth full-screen video playback
Build	Xcode project + GitHub Actions macOS runner (or Xcode Cloud) for CI/CD	Produces the signed .ipa automatically (see \$10)

4. Icons — What’s Coded vs. What’s AI-Generated (decision rule)

1. **Common UI glyph** (back chevron, search, bell, star, house, gear, calendar, phone, checkmark, location pin, camera, share, xmark, line.3.horizontal menu, pencil, trash, slider filter, chevron)? → Use **SF Symbols** (`Image(systemName: "...")`) — Apple’s real, official, built-in icon font, thousands of glyphs, supports multicolor/hierarchical rendering modes which look genuinely premium (`.symbolRenderingMode(.hierarchical)` or `.palette` with brand teal/orange). **Never** substitute an emoji character for these, and never export SF Symbols glyphs for use outside this iOS app (Apple’s usage terms restrict them to Apple-platform apps — the Android/Web builds use Material Symbols/Lucide instead, per the main brand blueprint).
2. **Brand-specific badge, category illustration, onboarding artwork, or empty-state illustration** with no simple glyph equivalent? → PNG asset, goes into `01_ai_images.json` (\$1.8).
3. **Simple geometric badge SwiftUI can draw directly** (plain circle with a checkmark, a colored dot, a progress ring via `Circle().trim()`)? → Draw it in SwiftUI directly — don’t spend an AI image slot on it.
4. **App icon** → built from the founder-provided master logo via Xcode’s asset catalog (\$1.7) — never AI-generated, never redrawn.

5. Splash Screen & Video Integration

1. iOS requires a static **Launch Screen** storyboard/`Info.plist`-based launch image for the very first instant frame (Apple does not allow a video here — this is an Apple platform rule, not a choice). Set this launch screen to a plain brand-teal (`#0D4A44`) background with the app icon/logo centered, so the transition into the video feels seamless rather than jarring.
2. Immediately after the app finishes launching, present a `SplashVideoView` (SwiftUI `UIViewControllerRepresentable` wrapping `AVPlayerViewController` with controls hidden, or a raw `VideoPlayer` from `AVKit`) that plays `Resources/SplashVideo.mp4` **full-screen edge-to-edge** (`.ignoresSafeArea()`), using `.videoGravity = .resizeAspectFill` (or `.resizeAspect` with brand-teal letterbox bars if the video’s native aspect ratio doesn’t match common iPhone screens — inspect the actual video once supplied and choose accordingly).
3. Plays exactly once, no loop, no visible controls. If longer than ~3-4 seconds, add a small “Skip” text button bottom-trailing after a 2-second delay, respecting safe-area insets.
4. On completion/skip, route to Onboarding (first launch, tracked via `UserDefaults/AppStorage`) or straight to the signed-in user’s Home/Dashboard.
5. The same master **app logo** asset must appear identically (one shared `AppLogo` SwiftUI view, never a separately-cropped copy) on: Login, Sign Up, “About Homigo Care” settings screen, and matches the App Icon exactly — visual consistency across every logo placement was explicitly required.

6. Authentication & OTP Flow (via the Admin Panel’s EmailJS SMTP)

Identical pattern to the Android app — the iOS app never talks to EmailJS directly:

1. User signs up (Email/Password, Google, or the iOS-exclusive **Sign in with Apple**) → Firebase Auth account created → app calls `POST /api/otp/send` on the Admin Panel API with the email → a 6-digit code is generated server-side, stored in Supabase `otp_codes` with a 5-minute expiry, and emailed via the Admin Panel’s EmailJS SMTP configuration.
2. App shows the 6-box OTP entry screen (S-08) → `POST /api/otp/verify` → on success, Admin Panel marks the Supabase profile `email_verified = true` → app proceeds to Profile Setup.
3. Google and Apple sign-ins skip this step (already-verified identity providers); Guest/Anonymous accounts are prompted to add an email later if they want to retain data across devices.
4. Phone-number SMS OTP is intentionally **not wired yet** — the phone field and its verification UI exist in the design for future use; only the Admin Panel’s backend implementation changes when Twilio/Firebase Phone Auth is

- switched on later, not this app’s screens.
5. Password reset follows the same /api/otp/send → 6-box entry → new password pattern.

If the Admin Panel’s OTP endpoints are not yet reachable/deployed when this phase is reached, **ask the founder for the live ADMIN_API_BASE_URL** rather than hardcoding a placeholder that silently fails.

7. Complete Screen-by-Screen Blueprint

Same screen set and same content as the Android app (one shared backend, one shared product) — every screen below is now expressed in native SwiftUI terms with the premium treatment from §2-§5. “Notes” only calls out anything iOS-specific.

7.1 Shared — Splash, Onboarding, Auth, Role Selection (9 screens)

ID	Screen	Flow	Content & Key Components (iOS-specific notes)
S-01	Splash (video)	Cold start → Onboarding (first launch) or Login	Static launch screen → full-screen AVPlayer video per §5
S-02	Onboarding 1/3	S-01 → S-03	TabView(.page) style paging, illustration
S-03	Onboarding 2/3	S-02 → S-04	il_onboarding_homecare (AI asset), headline, Skip/Next Illustration
S-04	Onboarding 3/3	S-03 → S-05	il_onboarding_verified (AI asset) Illustration
S-05	Login	S-04/logout → Home or S-06	il_onboarding_safety (AI asset), “Get Started” pill button
S-06	Sign Up	S-05 → S-07	AppLogo top-center, Email field, Password field (SecureField), “Sign in with Apple” button (Apple’s own SignInWithAppleButton, required alongside Google per App Store guidelines), “Continue with Google,” “Continue as Guest” text button, Sign Up link
S-07	Role Selection	S-06 → S-08 (Patient) or PR-03	Name, Email, Password, Confirm Password, Terms checkbox (CMS-fetched link), “Create Account”
S-08	OTP Verification	Per §6	Two large .continuous-corner cards: “I need care” vs “I provide care,” SF Symbol icons (heart.text.square / stethoscope)
S-09	Forgot Password	S-05 → S-08 (reset) → New Password	6-box native-feeling code entry (TextField per digit with auto-advance focus via @FocusState), “Resend code” 60s cooldown
			Email field, “Send Reset Code”

7.2 Patient Flow — 32 Screens (P-01-P-32)

ID	Screen	Flow	Content & Key Components
P-08	Who is this for?	S-08 (patient) → P-09	3 cards: Myself (Adult) / My Child / Elderly Family Member
P-09	Personal Details	P-08 → P-10	Name, CNIC + front/back photo (Cloudinary signed upload via PhotosPicker), DOB (DatePicker), Gender — Minor/Elderly variants collect the guardian’s CNIC instead
			MapKit map with “Use my current

P-10	Address & Emergency Contact	P-09 → P-11	location” (CLLocationManager), address fields, one required emergency contact
P-11	Home Dashboard	Entry point after setup	Large-title nav bar “Home,” greeting + bell icon (SF Symbol bell.badge), icon-based service grid using category badge PNGs, orange pulsing mic button for voice search (SFSpeechRecognizer), red “Book Emergency” banner pinned above the fold
P-12	Service Category	P-11 → P-13	Category sub-options with Budget/Premium price chips, presented as a .sheet or pushed UINavigationController destination
P-13	Browse Professionals	P-12 → P-14	List (List or LazyVStack in a ScrollView) of professional cards: photo, name, badge PNG, star rating, distance, price, “Book” button
P-14	Professional Profile	P-13 → P-15	Bio, hospital affiliation, badges, 5-category rating breakdown, moderated reviews, large-title nav with .navigationBarTitleDisplayMode(.large), “Book Now” sticky bottom button
P-15	Booking Details	P-14 → P-16/P-18	Booking type segmented control (Instant/Scheduled/Recurring/Online), DatePicker, notes field (text or voice-to-text), “Add to combined booking” toggle
P-16	Health Passport PIN Share	P-15 → P-17/P-18	PIN entry (SecureField, numeric), “Share Emergency Info Only” toggle, auto-revoke explanation text
P-17	Supply Pre-Request	P-16 → P-18	Supply item list with per-item “Approve Cost” checkboxes
P-18	Booking Summary & Payment	P-17 → P-19	Price breakdown, payment method picker (live from Admin Panel’s active payment_accounts), transaction ID field + optional screenshot upload for non-COD
P-19	Booking Confirmed	P-18 → P-11	Success illustration (AI asset), confirmation haptic (.notificationOccurred(.success)), booking reference, “View Booking”
P-20	Live Tracking	Near visit time → P-21	MapKit live annotation (Firestore-streamed position via AsyncStream), ETA, call/message buttons, Safe Zone radius overlay
P-21	Visit In Progress	P-20 → P-22	Status timeline, SOS button pinned bottom (exempt from scroll, always on top per safe-area rules)
P-22	Visit Complete	P-21 → P-23	Summary of visit notes, “Rate this visit” CTA
P-23	Rating & Review	P-22 → P-11	5-category HStack of SF Symbol stars (tap-to-rate), optional text review, anonymous toggle
P-24	Health Passport	Reachable from tab bar	Native TabView/segmented sub-tabs: Basic Info, Medical History, Documents, Activity Log, Vitals (native Swift Charts line graphs for BP/sugar/weight/temperature)
P-25	Emergency Screen	Reachable within 2 taps anywhere	Giant red Circle() SOS (3-second LongPressGesture), one-tap emergency numbers grid, works offline (cached locally in SwiftData/UserDefaults, SMS fallback via MFMessageComposeViewController if no data connection)
P-26	Lab Test Booking	From P-12 → P-18	Test/panel picker, partner lab selector, home-sample toggle
			Prescription upload or link to Health

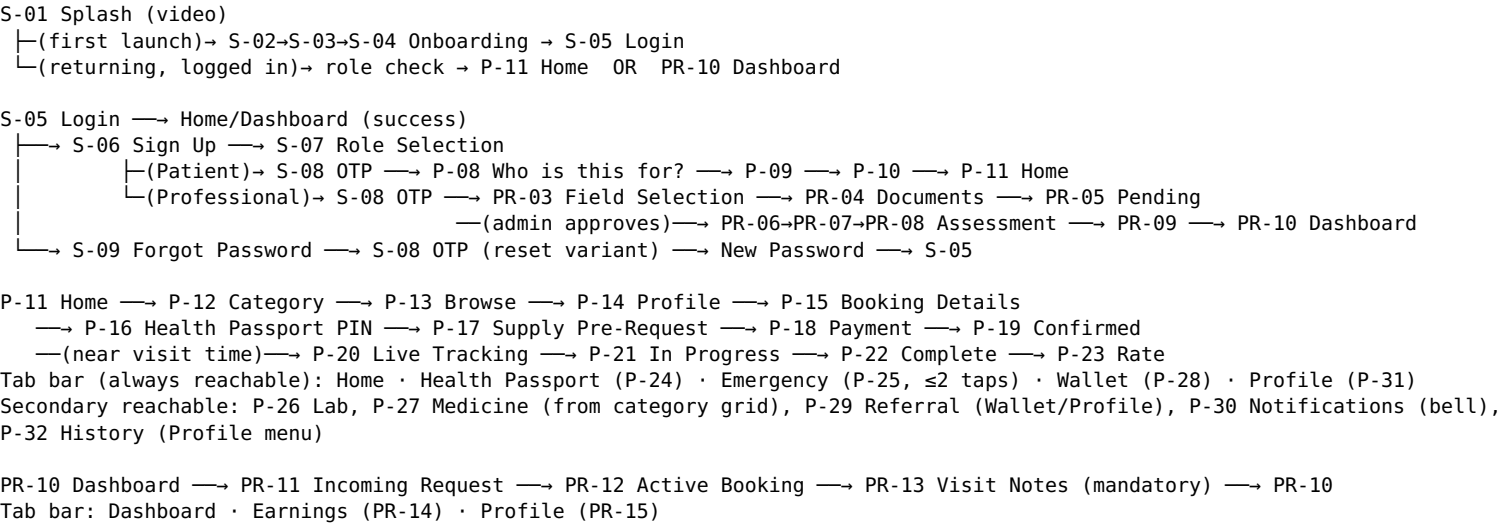
P-27	Medicine Order	From P-12 → P-18	Passport, pharmacy selector, delivery address confirm
P-28	Wallet	Tab bar	Balance, transaction history, “Top Up” with cashback tiers, referral code entry
P-29	Referral & Leaderboard	From P-28/Profile	Referral code + native ShareLink, live leaderboard, milestone progress bar
P-30	Notifications	Tab bar (badge count via <code>UNUserNotificationCenter</code>)	3 color-coded priority tiers, 30-day history
P-31	Profile & Settings	Tab bar	Profile edit, saved addresses, saved payment methods, language toggle (Urdu/English, full RTL layout mirroring via <code>.environment(\.layoutDirection, .rightToLeft)</code> when Urdu is active), quiet hours, About (CMS), Support (CMS), Delete Account (30-day soft delete), Sign Out
P-32	Booking History	Tab bar or Profile	Filterable list (Upcoming/Completed/Cancelled), tap → re-opens booking summary/rating

7.3 Professional Flow — 15 Screens (PR-01-PR-15)

ID	Screen	Flow	Content & Key Components
PR-03	Professional Field Selection	S-07 → PR-04	Category cards (Nursing Division, Medical & Dental, Allied Rehab & Diagnostics, Pharmacy Operations, etc.), then role list driven live from Supabase <code>role_requirements</code>
PR-04	Document Upload	PR-03 → PR-05	Dynamic form rendering exactly the required documents for the selected role plus CNIC front/back, each a Cloudinary signed-upload tile with live thumbnail
PR-05	Verification Pending	PR-04 → PR-06 (on admin approval, push-triggered)	Status illustration, review-time estimate, Support (CMS)
PR-06	Assessment Intro	Post-approval → PR-07	3-phase explanation, single-active-device warning, “Start Phase 1”
PR-07	Assessment — Live Test	PR-06 → PR-08	Server-driven countdown, one question per screen, no back-navigation, copy-paste disabled
PR-08	Assessment Results	PR-07 → PR-09 (pass) / retry (fail)	Score breakdown, badge-earned animation (Certified/Excellence/Elite)
PR-09	Profile Activation	PR-08 → PR-10	Availability hours, service radius, pricing tier, “Go Live” toggle
PR-10	Professional Dashboard	PR-09 → any below	Online/Offline toggle (prominent, top), today’s earnings, upcoming bookings, quick stats
PR-11	Incoming Booking Request	Push-triggered → PR-12/back	Booking details, 10-minute countdown ring (<code>Circle().trim()</code> animated), Accept/Reject
PR-12	Active Booking	PR-11 → PR-13	“Start Visit” (activates patient’s Safe Zone tracking), MapKit navigation, call/message, shared Health Passport PIN field if provided
			Vitals recorded, care

PR-13	Visit Notes (mandatory)	PR-12 → PR-14, cannot be skipped	checklist + free text, supply items used, photo attachment, “Complete Visit”
PR-14	Earnings & Payouts	Tab bar	Balance, payout request (Daily/Weekly/Monthly/On-demand, min PKR 500), transaction history, payout account entry
PR-15	Professional Profile Management	Tab bar	Edit bio/photo, badges, availability calendar, re-take assessment if prompted, Support (CMS), Sign Out

8. App-Wide Navigation Map (AppRouter.swift, NavigationStack-based)



Implement as one root NavigationStack per tab (standard iOS multi-tab pattern), with an AppRoute: Hashable enum driving .navigationDestination(for:) — this mirrors the Android app’s patientGraph/professionalGraph split conceptually, decided once at login via the role column, never re-checked mid-session.

9. Data Sync Architecture (iOS-side summary — identical backend to Android)

Data Type	Source	iOS Reads Via	iOS Writes Via
Login session	Firebase Auth	Auth.auth().currentUser	Firebase Auth SDK (+ Sign in with Apple credential bridge)
Live GPS / booking status / presence / chat	Firebase Firestore	addSnapshotListener wrapped in AsyncStream	Firestore setData/updateData
Push notifications	Firebase Cloud Messaging (via APNs)	UNUserNotificationCenterDelegate + MessagingDelegate	(Admin Panel triggers sends server-side)
Profiles, bookings ledger, reviews, payments, documents metadata, role requirements, wallet	Supabase Postgres	Supabase Swift SDK from("table").select(), RLS auto-scoped	Supabase insert/update, RLS-enforced
CNIC/degree/prescription/receipt files	Cloudinary (signed)	Signed URL fetched from Admin Panel API before display	Upload via signed preset after requesting a signature
App icon/illustration assets	Assets.xcassets (preferred) or Cloudinary homigo_public_assets	Image("name") or AsyncImage/Kingfisher	N/A — placed by founder per \$1.8
OTP send/verify, CMS content	Admin Panel Vercel API	URLSession/Alamofire calls to ADMIN_API_BASE_URL	N/A (read-only, except triggering OTP send)

This is the **same shared backend** the Android blueprint documents — the same Firebase project, the same Supabase database, the same Cloudinary account, the same Admin Panel. A booking made on Android and a booking made on iPhone both land in the same bookings table and are visible to the same Admin Panel in real time; there is no data silo

10. Build & Release Pipeline — Real Signed Build Requests

- `.github/workflows/ios-release.yml` runs on a **macOS GitHub Actions runner** (required — iOS builds cannot run on Linux/Windows runners):
 - Checkout → select Xcode version → resolve Swift Package dependencies → `xcodebuild` archive → export a signed `.ipa` using a distribution certificate + provisioning profile stored as **encrypted GitHub Actions secrets** (never committed).
 - Upload the `.ipa` as a workflow artifact so the founder can download and sideload it (via Xcode's Devices window, TestFlight, or ad-hoc distribution) to check on a real iPhone.
 - Honest cost note (cannot be made free, unlike everything else in this stack):** Apple requires an **Apple Developer Program membership (US \$99/year)** to generate the distribution certificate/provisioning profile needed for TestFlight or App Store distribution, and to register real test devices for ad-hoc builds. Without this membership, the app can only be run via a free personal Xcode signing on devices connected directly to a Mac, which is not practical for the founder's remote-build workflow described here. Budget for this membership before requesting the first real signed build.
 - The feedback loop the founder described (assets → agent → rebuild):** whenever the founder pushes newly generated PNGs (from `01_ai_images.json`) or an updated splash video/logo into the repository and tells the agent "these are ready," the agent must: (a) pull the new files, (b) wire each into its correct SwiftUI view/`Assets.xcassets` slot per the `destination_path` values already recorded, (c) commit, and (d) trigger the `ios-release.yml` workflow again (or instruct the founder how to re-trigger it) so a fresh, correctly-updated `.ipa` is produced for checking. This loop repeats every time new assets land — the agent should not require the founder to re-explain the wiring each time, since the destination paths were already fixed in §1.8.
 - Versioning: bump `CFBundleShortVersionString/CFBundleVersion` in `Info.plist` on every release-worthy push; keep `CHANGELOG.md` at the repo root, mirroring the Android repo's convention.
 - Point the debug/development scheme at `homigo-care-dev` Firebase config, and the release scheme at `homigo-care-prod`, via separate `GoogleService-Info.plist` files per scheme (Xcode build-configuration-based file selection) — this protects the **zero-fake-data production rule** while still allowing free testing in dev.
-

11. Final QA Checklist Before Calling the Build “Done”

- ☐ Every SwiftUI view uses only colors/fonts/shapes from `Core/Theme/` — zero hardcoded color literals or raw point sizes anywhere in `Features/`.
 - ☐ Every icon is either an SF Symbol, a SwiftUI-drawn shape, or a founder-approved AI PNG placed via §1.8 — no emoji used as icons.
 - ☐ “Sign in with Apple” is present alongside Google (App Store requirement when third-party login is offered).
 - ☐ Splash video plays full-screen exactly once with correct aspect handling, static launch screen shows first with no white flash.
 - ☐ App logo is identical (same asset, same shared `AppLogo` view) on Login, Sign Up, About screen, and the App Icon.
 - ☐ No screen clips content on iPhone SE (small) or a Pro Max (large) — tested at both sizes, safe-area insets respected everywhere, nothing sits under the Dynamic Island/home indicator.
 - ☐ SOS button (P-25) reachable within 2 taps from every screen, functions with cached emergency numbers even offline.
 - ☐ Health Passport (P-24) enforces the PIN gate for professional access and logs every access to `access_logs`.
 - ☐ Payment screen (P-18) lists only the **currently active** `payment_accounts` fetched live from Supabase — never hardcoded.
 - ☐ No table/collection contains seed/demo data — every list renders a genuine empty state against a fresh backend.
 - ☐ `assets_pending/01_ai_images.json` is valid JSON, 20–50 entries, every `destination_path` matches a real `Assets.xcassets` reference actually used in a SwiftUI view.
 - ☐ OTP send/verify round-trips correctly against the Admin Panel's real `/api/otp/*` endpoints (not mocked).
 - ☐ Confirmed the iOS app reads/writes the **same** Firebase project, Supabase database, and Cloudinary account as the Android app — a test booking created on one platform is visible from the other and from the Admin Panel.
 - ☐ A real signed `.ipa` was produced by the GitHub Actions macOS workflow and is downloadable as a build artifact.
-